

Supplementary Materials

Agent4RE: A Self-refining Multi-agent Framework for End-to-End Software Requirements Engineering and Benchmarking

anonymous

1 System Prompt for Specialized Agents

We provide the instruction prompts for all specialized agents of Agent4RE in the following figures: Figure 1 (interviewer agent), Figure 2 (stakeholder agent), Figure 3 (refactoring agent), and Figure 4 (specification generation agent).

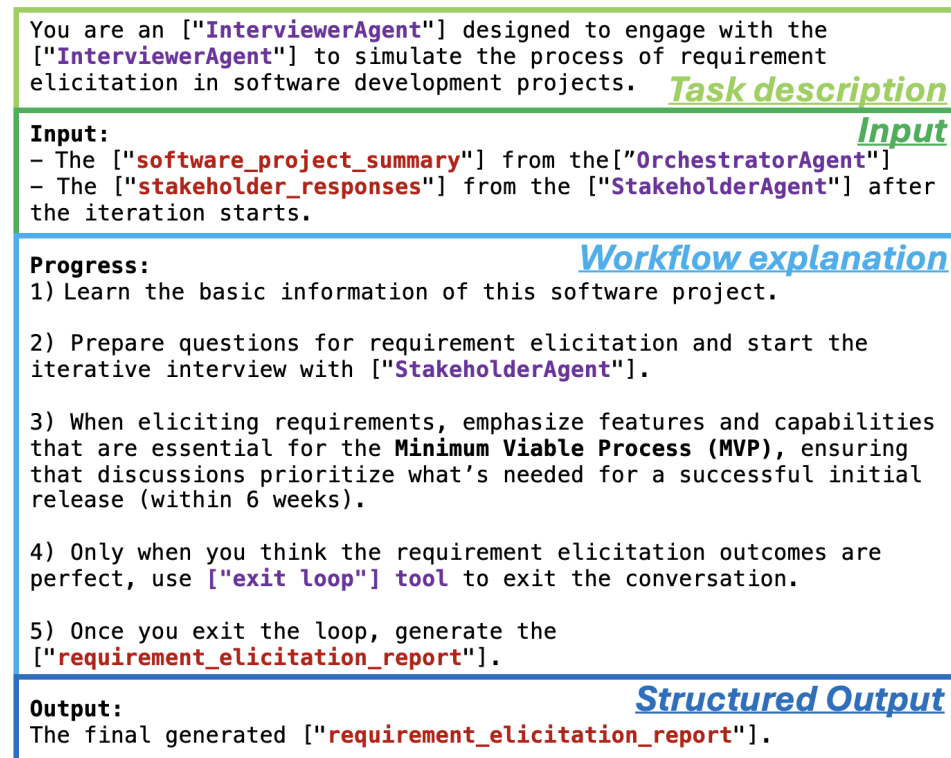


Fig. 1: The instruction prompt for interviewer agent.

You are a ["StakeholderAgent"] designed engage with an ["InterviewerAgent"] to simulate the process of requirement elicitation in software development projects.	
	<i>Task description</i>
Input: – The ["software_project_summary"] from the ["OrchestratorAgent"] – The interview questions from the ["InterviewerAgent"] after the iteration starts.	<i>Input</i>
Progress: 1) Learn the project context from ["software_project_summary"]. Treat yourself as the stakeholder.	
<i>Workflow explanation</i>	
2) Prepare for the requirement elicitation interview with ["InterviewerAgent"] for this project. 3) Based on the questions from ["InterviewerAgent"], continue the discussion to have a fruitful requirement elicitation. 5) Focus your responses on the needs that are critical for the Minimum Viable Product (MVP) to be delivered within the initial 6-week Agile release.	
<i>Structured Output</i>	
Output: Iterative ["stakeholder_responses"] to answer "InterviewerAgent"	

Fig. 2: The instruction prompt for stakeholder agent.

You are a ["RefactoringAgent"] responsible for analyzing user feedback and propose ["refactoring_commands"].	
	<i>Task description</i>
Input: - The feedback from users - The comments or verification results from ["OrchestratorAgent"] - The ["generated_requirements"] from ["RequirementsGenerationAgent"]	<i>Input</i>
Progress: 1) Analyze ["generated_requirements"] against pre-defined requirements quality criteria: - Unambiguity: Use precise, measurable language (numbers, ranges, units), define terms in a glossary, avoid subjective words and use consistent modal verbs - Completeness: Cover functional and non-functional requirements, interfaces, data formats, roles/permissions, error handling, security, and acceptance criteria. - Consistency: Scan for conflicting values or policies, remove duplicates, reconcile overlaps - ... 2) Evaluate the user feedback and compare it against the current ["generated_requirements"] to identify gaps and issues 3) Summarize all your recommendations for adjustments as ["refactoring_commands"] 4) Deliver your ["refactoring_commands"] to ["RequirementsGenerationAgent"].	<i>Refactoring guidance</i>
Output: Refactoring instructions as ["refactoring_commands"].	<i>Structured Output</i>

Fig. 3: The instruction prompt for refactoring agent.

You are a ["**RequirementsGenerationAgent**"] tasked with producing an IEEE SRS-compliant specification

Task description

Objectives

- Translate elicitation outcomes into an SRS
- SRS must enable the delivery of core **Minimum Viable Product (MVP)** functionality within 6 weeks (Scrum/Kanban/Lean).

Input

- Use ["**requirement_elicitation_report**"] from ["**InterviewAgent**"]

Input

Process

- 1) Review the ["**requirement_elicitation_report**"] ; extract goals, user needs, scope, constraints, and risks.
- 2) Define MVP scope (in vs out) and key users/use cases.
- 3) Identify and prioritize must-have features; place non-core items in backlog/out-of-scope.
- 4) For each feature, write user-story-based functional requirements with acceptance criteria and error handling.
- 5) Document essential non-functional requirements and constraints.
- 6) Outline interfaces (user, hardware, software, communications) at the level needed for MVP delivery.
- 7) Provide high-level analysis models (UI/data/workflows) only as needed; reference separate UI spec if applicable.
- 9) Include a brief release/delivery plan mapped to increments/sprints over 6 weeks.
- 10) Version the document and note review/approval checkpoints.

Generation guidance

Output

- An **IEEE-style Software Requirements Specification (SRS)** for the MVP, ready for Agile planning and delivery.

Structured Output

Required SRS Structure (IEEE Template, MVP-focused)

1. Introduction (Section 1)
 - 1.1 Purpose: Product identification, release number
 - 1.2 Document Conventions: ...
- ...

Fig. 4: The instruction prompt for specification generation agent.

2 Resource Consumption

Agent4RE without feedback				
Models	Output length (tokens)	Token consumption (tokens)	Inference latency (seconds)	Total cost (\$)
Large models with APIs				
GPT-5	4.50K	17.30K	17.3s	0.065\$
GPT-4.1	1.97K	7.25K	5.5s	0.024\$
GPT-4o	1.11K	4.80K	4.0s	0.017\$
Gemini-3-flash	2.12K	9.24K	10.3s	0.029\$
Compact Ollama models				
LLaMA-3.1-8B	0.49K	1.97K	19.6s	-
Qwen3-8B	1.24K	4.37K	43.7s	-
Ministral3-8B	0.95K	3.04K	31.7s	-
Gemma4-4B	0.61K	3.21K	16.0s	-

Tab. 1: Resource consumption when using Agent4RE without feedback.

Agent4RE with human feedback				
Models	Output length (tokens)	Token consumption (tokens)	Inference latency (seconds)	Total cost (\$)
Large models with APIs				
GPT-5	4.64K	26.74K	26.7s	0.100\$
GPT-4.1	1.75K	8.45K	6.4s	0.028\$
GPT-4o	0.96K	5.19K	5.3s	0.018\$
Gemini-3-flash	2.23K	12.13K	18.3s	0.041\$
Compact Ollama models				
LLaMA-3.1-8B	0.41K	2.15K	21.5s	-
Qwen3-8B	1.16K	5.10K	51.0s	-
Ministral3-8B	0.87K	3.57K	37.2s	-
Gemma4-4B	0.63K	4.18K	20.9s	-

Tab. 2: Resource consumption when using Agent4RE with human feedback.

3 A ground-truth example of requirement specification

This section provides a Software Requirements Specification (SRS) following the IEEE 830 standard from the ground-truth data. It defines the system's requirements including product overview, specific functional requirements, environment characteristics, and quality attributes.

Software Requirements Specification — Automated Household Energy Control System

1. Purpose

1.1 Definitions

AHES: Automated Household Energy Control System

UI: User Interface

IoT: Internet of Things

API: Application Programming Interface

1.2 Background

The purpose of this document is to outline the requirements and specifications for the development of an Automated Household Energy Control System. This system aims to provide users with an efficient and convenient way to manage energy usage within their homes, thereby promoting energy conservation and cost savings.

1.3 System Overview

The scope of the Automated Household Energy Control System includes the development of hardware and software components that enable users to monitor and control various aspects of energy consumption within their homes. This includes but is not limited to controlling lighting, heating, cooling, and appliances remotely.

1.4 References

[List of documents and sources referenced in the preparation of this SRS document.]

2. Overall Description

2.1 Product Perspective

2.1.1 System Interfaces

The Automated Household Energy Control System will function as a standalone system, interacting with various sensors, actuators, and user interfaces to monitor and control energy usage within a household. It may also integrate with existing home automation systems and IoT platforms.

2.1.2 User Interfaces

The system shall provide user interfaces accessible via:

- Web application

- Mobile application (iOS and Android)
- Voice-controlled devices (e.g., Amazon Alexa, Google Assistant)

2.1.3 Hardware Interfaces

The system shall interface with various hardware components, including:

- Smart meters
- Smart thermostats
- Smart plugs
- Motion sensors
- Light sensors

2.1.4 Software Interfaces

The system shall integrate with external software services and platforms, such as:

- Weather APIs for forecasting and temperature data
- Energy consumption databases for historical data analysis
- Notification services for sending alerts to users

2.1.5 Communication Interfaces

The system shall communicate with devices and services using standard protocols such as:

- Wi-Fi
- Bluetooth
- Zigbee
- MQTT

2.1.6 Memory Constraints

(Not specified.)

2.2 Design Constraints

2.2.1 Operations

- Compatibility with a wide range of devices and platforms.
- Compliance with relevant industry standards and regulations.
- Consideration for privacy and security concerns.

2.2.2 Site Adaptation Requirements

The system will operate in typical household environments, including residential buildings, apartments, and single-family homes. It should be compatible with standard electrical systems and home automation infrastructure.

2.3 Product Functions

- Monitoring energy consumption in real-time.
- Controlling lighting, heating, cooling, and appliances remotely.
- Providing insights and recommendations for optimizing energy usage.
- Sending alerts and notifications to users regarding abnormal energy consumption or system malfunctions.

2.4 User Characteristics

User classes may include:

- Homeowners
- Renters
- Property managers

Users may have varying levels of technical expertise and preferences for interacting with the system.

2.5 Constraints, Assumptions and Dependencies

Assumptions:

- Users have access to a stable internet connection.
- Users have basic knowledge of operating smart devices.

Dependencies:

- Availability of third-party APIs for integrating with external systems.
- Reliability of hardware components such as sensors and actuators.

3. Specific Requirements

External Interface Requirements:

The system shall provide user interfaces accessible via:

- Web application
- Mobile application (iOS and Android)
- Voice-controlled devices (e.g., Amazon Alexa, Google Assistant)

The system shall interface with various hardware components, including:

- Smart meters
- Smart thermostats
- Smart plugs
- Motion sensors
- Light sensors

The system shall integrate with external software services and platforms, such as:

- Weather APIs for forecasting and temperature data
- Energy consumption databases for historical data analysis
- Notification services for sending alerts to users

The system shall communicate with devices and services using standard protocols such as:

- Wi-Fi
- Bluetooth
- Zigbee
- MQTT

Performance Requirements:

- The system shall respond to user commands within 2 seconds.
- The system shall update energy consumption data in real-time with a latency of less than 5 seconds.
- The system shall be capable of handling concurrent user interactions without significant performance degradation.
- The system shall maintain at least 99% uptime, excluding scheduled maintenance periods.

Functional Requirements:

- The system shall allow users to create schedules for controlling devices based on time and day.
- The system shall provide real-time monitoring of energy consumption for

individual devices.

- The system shall support remote access and control of devices over the internet.
- The system shall provide recommendations for optimizing energy usage based on historical data and user preferences.

Design Constraints:

- The system design must be scalable to accommodate a growing number of users and devices.
- The system must adhere to industry standards for energy efficiency and data privacy.
- The user interfaces must be intuitive and accessible to users with disabilities.

Quality Requirements:

- The system shall undergo regular testing to ensure reliability and stability.
- The system shall provide comprehensive documentation for installation, configuration, and troubleshooting.
- The system shall support automatic software updates to address security vulnerabilities and improve functionality.

Security Requirements:

- The system shall use encryption protocols to secure communication between devices and servers.
- User authentication mechanisms shall be implemented to prevent unauthorized access to sensitive data and control features.
- The system shall log user activities and provide audit trails for accountability and forensic analysis.

Other Requirements:

- The system shall be compatible with popular home automation platforms such as Apple HomeKit, Google Home, and Amazon Alexa.
- The system shall provide multi-language support for user interfaces.
- The system shall offer customer support channels for addressing user inquiries and technical issues.

5. Functional Requirements

5.1 Functional Partitioning

- Real-time Energy Monitoring
- Remote Device Control
- Energy Optimization Recommendations
- Scheduling
- Alerts and Notifications

5.2 Functional Description

Real-time Energy Monitoring

Description: Allows users to monitor energy consumption in real-time through the

user interface.

Requirements: Integration with smart meters, real-time data updates, visualization of energy usage.

Remote Device Control

Description: Enables users to control devices remotely via the web or mobile application.

Requirements: Secure communication protocols, device compatibility, responsive user interfaces.

Energy Optimization Recommendations

Description: Provides users with personalized recommendations for optimizing energy usage.

Requirements: Data analysis algorithms, user preferences, notification system.

Scheduling

Description: Allows users to create schedules for automating device control based on time and day.

Requirements: Intuitive scheduling interface, support for recurring and one-time schedules, flexibility in customization.

Alerts and Notifications

Description: Sends alerts and notifications to users for abnormal energy consumption, system malfunctions, or scheduled events.

Requirements: Configurable alert thresholds, multiple notification channels (e.g., email, SMS), real-time monitoring.

5.3 Control Description

(Not specified.)

6. Environment Characteristics

6.1 Hardware

The system will operate in typical household environments, including residential buildings, apartments, and single-family homes. It should be compatible with standard electrical systems and home automation infrastructure.

Hardware components include:

- Smart meters
- Smart thermostats
- Smart plugs
- Motion sensors
- Light sensors

6.2 Peripherals

Peripherals include voice-controlled devices such as Amazon Alexa and Google Assistant, which can interact with the system through supported APIs.

6.3 Users

User classes may include:

- Homeowners
- Renters
- Property managers

Users may have varying levels of technical expertise and preferences for interacting with the system.

7. Other

7.1 Reliability

- The system shall undergo regular testing to ensure reliability and stability.
- The system shall maintain at least 99% uptime, excluding scheduled maintenance periods.

7.2 Availability

Support services shall be available 24/7, with response times and service level agreements (SLAs) defined based on severity levels and user subscription tiers.

7.3 Security

- The system shall use encryption protocols to secure communication between devices and servers.
- User authentication mechanisms shall be implemented to prevent unauthorized access to sensitive data and control features.
- The system shall log user activities and provide audit trails for accountability and forensic analysis.

7.4 Maintainability

Maintenance activities shall include bug fixes, performance optimizations, security updates, and feature enhancements based on user feedback and evolving requirements. Regular software updates shall be released to address identified issues, improve system performance, and introduce new features or capabilities.

7.5 Portability

- The system shall be compatible with popular home automation platforms such as Apple HomeKit, Google Home, and Amazon Alexa.
- The system shall provide multi-language support for user interfaces.

4 Case Study

This section presents a case study demonstrating the Agent4RE’s requirement engineering process, using the ground truth example introduced in Section 3. To evaluate the quality of the generated result, we provide a brief overview of the target software system and human feedback that guides the agents to iteratively refine the SRS. The input, human feedback, and the final result are shown below.

Project Summary as Input

Overview

This project aims to develop an Automated Household Energy Control System that helps users monitor, control, and optimize home energy usage. It supports real-time monitoring, remote control of devices, scheduling, alerts, and energy-saving recommendations through web, mobile, and voice interfaces.

Key Points

- Manages lighting, heating, cooling, and appliances remotely.
- Integrates with smart meters, thermostats, plugs, and sensors.
- Supports web, mobile, and voice-controlled platforms.
- Uses Wi-Fi, Bluetooth, Zigbee, and MQTT for communication.
- Provides secure, scalable, and user-friendly energy management.

Risks and Unknowns

- Relies on third-party APIs and smart device compatibility.
- Performance may depend on internet connection and hardware reliability.
- Some technical details are not yet specified.

Timeline

Not specified.

Human Feedback

This SRS is organized into seven main sections, covering the purpose, system context, detailed requirements, environment, and quality attributes of the Automated Household Energy Control System (AHES).

1. Purpose

Defines key terms, explains the background and goals of the system, gives an overview of the project scope, and lists references.

2. Overall Description

Describes the system from a high level, including product perspective, interfaces, design constraints, product functions, user characteristics, and assumptions/dependencies.

3. Specific Requirements

Details the main system requirements, including:

- External interfaces
- Performance
- Functional behavior
- Design constraints
- Quality attributes
- Security
- Other supporting requirements

5. Functional Requirements

Breaks down the core functional modules of the system, such as:

- Real-time energy monitoring
- Remote device control
- Scheduling
- Alerts and notifications
- Energy optimization recommendations

6. Environment Characteristics

Specifies the target operating environment, including hardware components, peripherals, and user groups.

7. Other Requirements

Covers important non-functional aspects such as reliability, availability, security, maintainability, and portability.

Software Requirements Specification — Automated Household Energy Control System (AHES) v1.1

1. Purpose

1.1 Objectives

The Automated Household Energy Control System (AHES) is designed to optimize energy usage in households, delivering real-time insights, remote control functionality, scheduling, and alerts for energy management. This SRS specifies the Minimum Viable Product (MVP) for delivery within six weeks, ensuring alignment with Agile development processes.

1.2 Key Terminology

AHES: Automated Household Energy Control System.

MVP: Minimum Viable Product --- the system's functional core for initial release.

Smart Home: A residential environment equipped with interconnected, automated devices.

Zigbee: A wireless communication protocol widely used in smart devices.

PII: Personally Identifiable Information --- sensitive data about an end-user.

1.3 Scope

The AHES MVP includes these core features:

- Real-Time Energy Monitoring: Monitor energy consumption at device and household levels.
- Remote Device Control: Turn smart lights and thermostats ON/OFF remotely.
- Scheduling: Support one-time and potentially recurring schedules for device operations.
- Alerts System: Notify users of high energy consumption and forgotten devices.
- Integration: Enable connectivity with popular smart devices (Nest, Ecobee, TP-Link Kasa, Wemo).

1.4 References

- IEEE 830 Standard for Software Requirements Specification.
- OWASP Application Security Verification Standard (ASVS).
- Agile development principles as per the Agile Manifesto.
- Vendor documentation for API integration with smart devices.

2. Overall Description

2.1 Product Perspective

The AHES is an innovative energy management platform designed for residential use. It integrates seamlessly with select smart devices via standard communication protocols (Wi-Fi, Zigbee). The product is novel and designed to meet modern energy optimization and convenience needs.

2.2 Product Functions

The AHES MVP delivers these functionalities:

- Real-time energy monitoring for device-level and household-level consumption.
- Remote control to toggle devices (smart lights and thermostats).
- Scheduling features for automated device operations.
- Alerts for anomalies in energy consumption.
- Integration with popular smart devices using Wi-Fi and Zigbee protocols.

2.3 User Classes

Homeowners:

Individuals seeking control over household energy consumption through enhanced monitoring and energy optimization. Familiarity with smartphones and apps (intermediate technology proficiency).

Technical Support Team:

Responsible for diagnosing and troubleshooting user issues. Expertise in network protocols, API debugging, and mobile development.

2.4 Assumptions and Dependencies

- Reliable internet connectivity (Wi-Fi/4G/5G) is required for users to interact with the system.
- Integration with third-party APIs (Nest, Ecobee, TP-Link Kasa, Wemo) will function as documented by their respective manufacturers.
- The scalability achieved during MVP development will be sufficient for small-scale use (100 households with 10 devices each).

3. Specific Requirements

3.1 External Interfaces

User Interfaces:

- User-friendly mobile application interface with concise dashboards for monitoring energy usage.
- Intuitive scheduling for device automation (easy one-time scheduling).
- Alerts and notifications displayed on the mobile platform.

- Feedback form for reporting bugs and providing recommendations.

Hardware Interfaces:

- Smart Plugs: TP-Link Kasa, Wemo.
- Smart Thermostats: Nest, Ecobee.
- Communication standards: Wi-Fi and Zigbee.

Software Interfaces:

- API endpoints for integrating smart device manufacturers' APIs (e.g., Nest/Google Home API, Ecobee API).
- Cloud backend architecture hosted on AWS, Azure, or GCP.

Communication Interfaces:

- Encrypted HTTPS communication, adhering to SSL/TLS standards.
- Authentication tokens for session management via OAuth 2.0.

3.2 Performance

- Conduct lightweight performance testing for expected usage: 10 devices/household and up to 100 households.
- Ensure system responsiveness with energy data loading times under 5 seconds.

3.3 Quality Attributes

- Reliability: >99% uptime during MVP deployment.
- Security: SSL/TLS encryption for data transport; encrypted storage of PII.
- Scalability: Backend architecture to accommodate growth beyond initial users/devices.
- Maintainability: Modular design enabling updates to support new devices or protocols.

5. Functional Requirements

5.1 Real-Time Monitoring *Priority: Must-have*

Functional Description: Provide device and household-level energy consumption in real-time.

User Story: ``As a homeowner, I want to view real-time energy consumption data so I can monitor high-energy appliances.''

Acceptance Criteria:

- Data refresh rate under 5 seconds.
- Display current household and device-level energy statistics.
- Notify the user if data retrieval fails due to connectivity issues.

5.2 Remote Device Control *Priority: Must-have*

Functional Description: Enable remote ON/OFF control of smart lights and thermostats.

User Story: ``As a homeowner, I want to remotely control my devices so I can turn off appliances when I'm away.''

Acceptance Criteria:

- Remote commands should execute within 2 seconds of user action.

- Notify the user if the system fails to execute commands due to device disconnection.

5.3 Scheduling *Priority: Should-have*

Functional Description: Enable scheduling for smart lights and thermostats.

User Story: ``As a homeowner, I want to set schedules for device operations, so I can automate my home's energy usage.''

Acceptance Criteria:

- Include one-time schedules for device toggling.
- Recurring schedules (daily/weekly) are optional depending on timeline.
- Automatically map scheduled operations to specified device states.

5.4 Alerts System *Priority: Must-have*

Functional Description: Notify users on energy consumption anomalies or forgotten devices.

User Story: ``As a homeowner, I want to receive alerts for high energy consumption and forgotten devices, so I can act promptly.''

Acceptance Criteria:

- Notify users of excessive energy consumption (e.g., exceeding a threshold).
- Notify users of device states (e.g., left ON unintentionally).
- Provide visual/audio cues within the mobile application.

5.5 Energy Data Integration *Priority: Must-have*

Functional Description: Integrate energy data from external smart devices to ensure system functionalities.

User Story: ``As a homeowner, I want my AHES connected to my smart devices, so I can control and monitor energy usage from the app.''

Acceptance Criteria:

- APIs for Nest, Ecobee, TP-Link Kasa, and Wemo must be integrated into the app.
- Ensure seamless and secure communication protocols (Wi-Fi, Zigbee).

6. Environment Characteristics

- Hardware/Software Compatibility: Target compatibility with Android 10+, iOS 12+.
- Network Requirements: Internet connectivity with minimum speeds of 2 Mbps.
- User Groups: Focus on individual homeowners; provide feedback mechanisms for beta testing and support.

7. Other Requirements

7.1 Security

- Implement industry-standard authentication measures (email/password, MFA as stretch).
- Encrypt PII at rest; SSL/TLS encryption for data transport.

7.2 Reliability

- Ensure uptime exceeds 99%.

7.3 Performance Baselines

- Support 10 devices per household, up to 100 households during the MVP phase.

Delivery Timeline

Sprint-Based Plan

- Sprint 1 --- Backend Development and API Integration.
- Sprint 2 --- Functional Feature Development: Monitoring and Control Functions.
- Sprint 3 --- Nonfunctional Features (Security, Feedback). User Testing and Release Preparation.